Introduction

What is a Hook?

- A Hook is a configuration that says: "When event X fires, run this command.
- A hook is a user-defined **shell command** that runs at a specific point in Claude Code's lifecycle.
- Hooks fire on file edits, tool calls, session start, prompt submission, compaction, and many more events.
- Hooks give you **deterministic control** over Claude Code's behaviour — they ensure certain actions always happen, instead of relying on the language model to choose to run them.

Skills vs Hooks: Skills are advisory; hooks are enforced

- You can put "always run **npm test** before committing" in a SKILL.md, and Claude will probably do it. But the model has the final say.
- A **PreToolUse** hook on Bash that blocks any commit when tests fail is non-negotiable — Claude literally cannot proceed past it.

	INSTRUCTION FILE OR PROMPT	**HOOK**
What it is	A request the model reads	Code the harness runs
Can the model skip it?	Yes (it usually obeys, not always)	No
Can it stop a tool call?	No	Yes (PreToolUse)
Costs context?	Yes	Only what the hook prints
Best for	Style, explanations, judgement	"Always" and "never"

What hooks unlock

- **Formatters and linters:** Prettier, ESLint, Black, gofmt **run** automatically after every file edit.
- **Guardrails:** Block edits to .env, package-lock.json, .git/, or anything else that should not be touched.
- **Audit trails:** Append every Bash command, every config change, every tool call to a log for compliance.
- **Notifications:** Desktop ping when Claude is waiting for permission so you can switch to other tasks.
- **Context injection:** Re-add project rules after compaction so Claude doesn't forget conventions.
- **Auto-approval:** Skip permission dialogs for tool calls you always allow.
- **Environment management:** Reload direnv variables when Claude changes directory.

Events List

Hook events fire at specific lifecycle points. When an event fires, all matching hooks **run in parallel** and identical commands are deduplicated. Below is the complete current event catalogue from the official documentation.

Session lifecycle events

Event	When it fires
SessionStart	When a session begins or resumes
Setup	When you start with --init-only, or with --init/--maintenance in -p mode
SessionEnd	When a session terminates
CwdChanged	When the working directory changes (e.g. Claude runs cd)
FileChanged	When a watched file changes on disk; matcher names the files
ConfigChange	When a settings or skills file changes during a session
InstructionsLoaded	When CLAUDE.md or .claude/rules/*.md is loaded into context

Prompt and tool events

Event	When it fires
UserPromptSubmit	When you submit a prompt, before Claude processes it
UserPromptExpansion	When a typed command expands into a prompt; can block the expansion
PreToolUse	Before a tool call executes; can block it
PostToolUse	After a tool call succeeds
PostToolUseFailure	After a tool call fails
PostToolBatch	After a full batch of parallel tool calls resolves, before the next model call
PermissionRequest	When a permission dialog appears
PermissionDenied	When a tool call is denied by the auto-mode classifier
Notification	When Claude Code sends a notification (idle, permission prompt, etc.)

Agent and task events

Event	When it fires
SubagentStart	When a subagent is spawned
SubagentStop	When a subagent finishes
TaskCreated	When a task is created via TaskCreate
TaskCompleted	When a task is marked completed
WorktreeCreate	When a worktree is created via --worktree or isolation: 'worktree'
WorktreeRemove	When a worktree is removed
Stop	When Claude finishes responding
StopFailure	When a turn ends due to an API error

Compaction and MCP events

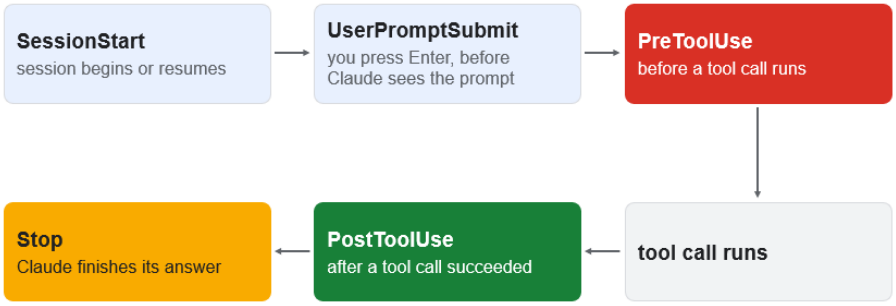
Event	When it fires
PreCompact	Before context compaction
PostCompact	After context compaction completes
Elicitation	When an MCP server requests user input during a tool call
ElicitationResult	After a user responds to an MCP elicitation, before the response is sent back

Mental model

Pick the event that gives you the earliest correct enforcement point.

- PreToolUse blocks before damage;
- PostToolUse reacts after.
- SessionStart sets up;
- SessionEnd cleans up.
- UserPromptSubmit can inject context every turn.

Knowing these moments unlocks every hook recipe.



Anatomy of a Hook Configuration

Hooks are configured as JSON.

```
{
 "hooks": {
  "<EventName>": [
   {
    "matcher": "<optional pattern>",
    "hooks": [
     {
      "type": "command",
      "command": "<shell command>",
      "timeout": 60,
      "if": "<optional permission-rule pattern>"
     }
    ]
   }
  ]
 }
}
```

Three nested levels:

1. Top-level hooks object — keyed by event name (PreToolUse, PostToolUse, SessionStart, ...).
2. Array of handler groups — each can have its own matcher to filter when it fires.

3. Inner hooks array — the actual list of commands to run when the matcher matches.

Where Hooks Live:

Location	Scope	Shareable
~/.claude/settings.json	All your projects on this machine	No — local to your machine
.claude/settings.json	Single project (committed to repo)	Yes — every contributor gets it
.claude/settings.local.json	Single project (gitignored)	No — your local overrides
Managed policy settings	Whole organisation	Yes — admin-controlled, users cannot disable
Plugin hooks/hooks.json	Whoever enables the plugin	
Skill or subagent frontmatter	Only while that skill or subagent is active	

Note: Hooks from all levels merge; they do not replace each other

Build Your First Hook

We'll set up a desktop notification hook that pings you whenever Claude Code is waiting for input — so you can switch tasks instead of staring at the terminal. Total time: about two minutes.

Step 1 — Open user settings

```
# macOS / Linux
nano ~/.claude/settings.json


# Windows (PowerShell)
notepad $HOME\.claude\settings.json
```

Step 2 — Add the Notification hook

Pick the variant that matches your OS:

macOS:

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
```

```
        "command": "osascript -e 'display notification \"Claude Code needs your attention\" with title
\"Claude Code\"'"
       }
      ]
    }
  ]
 }
}
```

Windows (PowerShell):

```
{
  "hooks": {
    "Notification": [
     {
       "matcher": "",
       "hooks": [
        {
          "type": "command",
          "command": "powershell.exe -Command
\"[System.Reflection.Assembly]::LoadWithPartialName('System.Windows.Forms');
[System.Windows.Forms.MessageBox]::Show('Claude Code needs your attention', 'Claude Code')\""
        }
       ]
     }
    ]
  }
}
```

Step 3 — Verify

In a Claude Code session, run /hooks. You should see Notification listed with a count of 1. Selecting it shows the matcher, command, and source file.

Step 4 — Test

Ask Claude to do something that needs permission (e.g. "run npm install"). Switch away from the terminal. When Claude is waiting, a desktop notification appears.

Filtering with Matchers and the if Field

Without a matcher a hook fires every time its event happens. Matchers narrow that down.

MATCHER	MATCHES
"*", "" or omitted	Everything
Bash	Exactly the Bash tool
Edit|Write	Either tool (letters, digits, _, -, ,, | only = exact list)
mcp__github__.*	Every tool of the github MCP server (any other character makes it a regex)
Edit.*	Regex, **unanchored**: also matches NotebookEdit. Use ^Edit$ for exactness

Per-event matcher behaviour

Event family	Matcher filters	Example values
PreToolUse, PostToolUse, PostToolUseFailure, PermissionRequest, PermissionDenied	Tool name	Bash, Edit|Write, mcp__.*
SessionStart	How session started	startup, resume, clear, compact
Setup	Which CLI flag triggered setup	init, maintenance
SessionEnd	Why session ended	clear, resume, logout, prompt_input_exit, ...
Notification	Notification type	permission_prompt, idle_prompt, auth_success, ...
SubagentStart, SubagentStop	Agent type	general-purpose, Explore, Plan, custom names
PreCompact, PostCompact	What triggered compaction	manual, auto
ConfigChange	Configuration source	user_settings, project_settings, local_settings, policy_settings, skills
FileChanged	Filenames to watch (literal, pipe-separated)	.envrc|.env
UserPromptExpansion	Command name	your skill or command names
UserPromptSubmit, PostToolBatch, Stop, ...	No matcher support	always fires

The if field — filter by tool arguments

- Matcher filters by tool name. The **if field** goes one step further and filters by arguments using the same syntax as permission rules.
- It is supported **on tool events only** and requires Claude Code v2.1.85+.

Run a hook only on git Bash commands:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "if": "Bash(git *)",
            "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/check-git-policy.sh"
          }
        ]
      }
    ]
  }
}
```

The rules:

- **One rule per hook.** No &&, no lists.
- For **Bash**, **each sub-command of a compound command is checked**. So if: "Bash(git push *)" also fires for cd x && git push.
- **It filters, it does not enforce.** If Claude Code cannot tell what the command is, **the hook runs anyway**.

Hook Input and Output

Command hooks communicate with Claude Code through three channels: **stdin** (event data in), **stdout** (decisions or context out), **stderr** (error messages or block reasons), plus an **exit code** that controls flow.

What lands on stdin?

Claude Code pipes JSON to your script's stdin. Common fields appear on every event:

```
{
  "session_id": "abc123",
  "cwd": "/demo/myproject",
  "hook_event_name": "PreToolUse"
```

```
}
```

Each event adds its own data. A **PreToolUse** hook on a Bash call gets:

```
{
  "session_id": "abc123",
  "cwd": "/demo/myproject",
  "hook_event_name": "PreToolUse",
  "tool_name": "Bash",
  "tool_input": {
    "command": "npm test"
  }
}
```

UserPromptSubmit hooks receive the prompt text. SessionStart hooks receive a source field (startup, resume, clear, compact). And so on.

tool_input fields by tool:

TOOL	TOOL_INPUT FIELDS
Bash	command, description, timeout, run_in_background
Read	file_path (absolute; backslashes on Windows), offset, limit
Edit	file_path, old_string, new_string, replace_all
Write	file_path, content
Grep	pattern, path, glob, output_mode

Exit codes drive the simple decisions

Exit code	Meaning
0	Action proceeds. For UserPromptSubmit / UserPromptExpansion / SessionStart, anything written to stdout is added to Claude's context.
2	Action is blocked. stderr becomes Claude's feedback so it can adjust. Some events ignore exit 2 (SessionStart, Setup, Notification, ...) — for those, stderr just shows to the user and execution continues.
Other	Action proceeds. Transcript shows '<hook> hook error' followed by first line of stderr. Full stderr goes to the debug log.

Two ways to block or to decide.

1. Exit 2 plus a stderr sentence:
    - The simplest block.
    - stderr is the reason, shown to Claude.

- Exit 2 blocks even if the same hook printed an "allow".

2. JSON with exit 0:
   - For a precise decision, a rewrite, added context, or a non-blocking message.
   - stdout must be one JSON object and nothing else.
   - Plain text on stdout is ignored, except for UserPromptSubmit, UserPromptExpansion, SessionStart and PostModelSwitch, where it is added to Claude's context.

Hook output on Stdout with exit code 0

```
{
 "hookSpecificOutput": {
  "hookEventName": "PreToolUse",
  "permissionDecision": "deny",
  "permissionDecisionReason": "Production config is read-only"
 }
}
```

The permissionDecision can be:

- deny - the call does not run. The reason goes to Claude.
- ask - force a permission prompt.
- allow - skip the prompt. But deny and ask rules in settings still apply.
- defer - leave the decision to the normal flow.

Rule: do not mix. JSON is ignored when the exit code is 2.

Example: Block dangerous SQL

A hook has two parts:

1. **The script** — the .sh file you have above. It receives the tool-call data on stdin, decides what to do, and signals back via exit code.
2. **The registration** — a JSON entry in your settings file that tells Claude Code *when* to run the script.

1. Create file .claude/hooks/block-drops.sh

```
#!/bin/bash
INPUT=$(cat)

# To learn about the json input format
echo "$INPUT" >> "$CLAUDE_PROJECT_DIR/.claude/hooks/hook-input.log"
echo "---" >> "$CLAUDE_PROJECT_DIR/.claude/hooks/hook-input.log"

# Try common field names across Bash/PowerShell tool schemas
```

```
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // .tool_input.script // .tool_input.code //
empty')

if echo "$COMMAND" | grep -iq "drop" && echo "$COMMAND" | grep -iq "table"; then
 echo "Blocked: dropping tables is not allowed" >&2
 exit 2
fi
exit 0
```

2. Install jq

jq is a command-line JSON processor. Think of it as grep for JSON, but structurally aware. The -r flag stands for raw output:

https://jqlang.org/download/

3. Update settings.json

```
"hooks": {
 "PreToolUse": [
  {
   "matcher": "Bash|PowerShell",
   "hooks": [
    {
     "type": "command",
     "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/block-drops.sh"
    }
   ]
  }
 ]
}
```

4. Ask claude to first create a table and then test using drop the same table.

Hook Types		
TYPE	**WHAT RUNS**	**USE IT FOR**
command	A program or script; event JSON on stdin	Almost everything. The default choice
http	POST of the event JSON to a URL; response uses the same output format	A central policy or audit service

mcp_tool	A tool on a connected MCP server	When the check already exists as an MCP tool
prompt	A single model call that judges the event	Fuzzy yes/no judgements

Examples:

Step 1: .claude\settings.json

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "http",
            "url": "http://localhost:5000/hooks/check-command",
            "timeout": 10
           "headers": {
                   "Authorization": "Bearer $HOOK_API_TOKEN"
               },
          }
        ]
      }
    ]
  }
}
```

Step2: Hookserver.py

```
from flask import Flask, request, jsonify
app = Flask(__name__)


@app.post("/hooks/check-command")
def check_command():
  # Claude Code sends the hook event as JSON.
  payload = request.get_json(silent=True) or {}

  # For a Bash tool call, the command is inside tool_input.
  tool_input = payload.get("tool_input") or {}
  command = tool_input.get("command", "")
```

```
  print(f"Claude wants to run: {command}", flush=True)

  # Demonstration: deny commands containing this exact text.
  if "rm -rf" in command.lower():
    return jsonify({
      "hookSpecificOutput": {
        "hookEventName": "PreToolUse",
        "permissionDecision": "deny",
        "permissionDecisionReason": (
          "Commands containing 'rm -rf' are blocked "
          "by the demo policy."
        )
      }
    })

  # Make no permission decision. Normal Claude Code permission
  # handling continues.
  return jsonify({})


if __name__ == "__main__":
  app.run(host="127.0.0.1", port=5000)
```

Step 3: Runt the following commands

```
pip install flask
python hookserver.py
```

Step 4: Test endpoint directly:

```
curl -X POST http://127.0.0.1:5000/hooks/check-command \
  -H "Content-Type: application/json" \
  -d '{"tool_input":{"command":"rm -rf test-folder"}}'
```

Then start Claude Code from your project and ask it to run a harmless command, such as `ls -la`. You should see the command printed in the Flask terminal.

```
List the files in the current directory using a Bash command.
```

Note: The string check is only for demonstration and does not catch all dangerous commands. Also, if the API is unavailable or times out, HTTP hook errors are non-blocking, so the command may still proceed through normal permission handling.

Managed settings

- **allowManagedHooksOnly**: **true** - only managed hooks run. User, project, local and plugin hooks are blocked.
- **disableAllHooks**: **true** - switches all non-managed hooks off.
- **allowedHttpHookUrls** - limits where http hooks may post.

settings.json

```
{
  "disableAllHooks": false,
  "allowedHttpHookUrls": [
    "http://127.0.0.1:5000/hooks/check-command"
  ],
}
```

Common Examples

Each recipe below is a copy-paste-ready settings block. Drop them into ~/.claude/settings.json (user) or .claude/settings.json (project) and Claude Code will pick them up.

1. Auto-format after every edit

```
{
 "hooks": {
  "PostToolUse": [
   {
    "matcher": "Edit|Write",
    "hooks": [
     {
      "type": "command",
      "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
     }
    ]
   }
  ]
 }
}
```

Replace prettier with eslint --fix, black, gofmt, dotnet format, or your team's formatter.

2. Block edits to protected files

A two-piece recipe: a script and a hook entry that calls it.

File: **.claude/hooks/protect-files.sh**

```
#!/bin/bash
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

PROTECTED_PATTERNS=(".env" "package-lock.json" ".git/")

for pattern in "${PROTECTED_PATTERNS[@]}"; do
 if [[ "$FILE_PATH" == *"$pattern"* ]]; then
  echo "Blocked: $FILE_PATH matches protected pattern '$pattern'" >&2
  exit 2
 fi
done
exit 0
```

For MaxOS : Make it executable:

```
chmod +x .claude/hooks/protect-files.sh
```

Register it in .claude/settings.json:

```
{
 "hooks": {
  "PreToolUse": [
   {
    "matcher": "Edit|Write",
    "hooks": [
     {
      "type": "command",
      "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/protect-files.sh"
     }
    ]
   }
  ]
 }
}
```

3. Re-inject context after compaction

When the context window fills up, compaction summarises the conversation and important details may be lost. A SessionStart hook with a compact matcher restores them.

```
{
  "hooks": {
    "SessionStart": [
      {
        "matcher": "compact",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Reminder: use Bun, not npm. Run bun test before committing. Current sprint: auth refactor.'"
          }
        ]
      }
    ]
  }
}
```

Anything written to stdout from a SessionStart, UserPromptSubmit, or UserPromptExpansion hook is added to Claude's context. Replace the echo with git log --oneline -5 or any command that emits relevant state.

4. Audit configuration changes

```
{
  "hooks": {
    "ConfigChange": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "jq -c '{timestamp: now | todate, source: .source, file: .file_path}' >> ~/claude-config-audit.log"
          }
        ]
      }
    ]
```

```
  }
}
```

5. Auto-approve harmless permission prompts

Skip the dialog for tool calls you always allow. This example auto-approves ExitPlanMode (the prompt that asks 'plan looks good — proceed?'):

```
{
  "hooks": {
    "PermissionRequest": [
      {
        "matcher": "ExitPlanMode",
        "hooks": [
          {
            "type": "command",
            "command": "echo '{\"hookSpecificOutput\": {\"hookEventName\": \"PermissionRequest\",
\"decision\": {\"behavior\": \"allow\"}}}'"
          }
        ]
      }
    ]
  }
}
```

Keep matchers narrow

An empty matcher or .* on PermissionRequest auto-approves every prompt — including file writes and shell commands. Always scope to specific tool names you've vetted.

End-to-End Example: A Compliance Hook Bundle

Let's build a real bundle that combines four hooks: format on edit, block secrets, audit Bash commands, and re-inject project rules after compaction.

These four hooks together encode roughly 80% of what teams need from Claude Code automation:

1. Secret protection
2. Formatting
3. Audit Bash / Powershell Commands,
4. Convention Reinforcement.

Step 1 — block-secrets.sh

```
#!/bin/bash
```

```
# .claude/hooks/block-secrets.sh
# Blocks edits to secret-bearing files and rejects content that contains obvious credentials.

INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')
CONTENT=$(echo "$INPUT" | jq -r '.tool_input.content // .tool_input.new_string // empty')

# 1. Path-based block
PROTECTED=(".env" "secrets/" "credentials.json" "id_rsa" ".pem")
for p in "${PROTECTED[@]}"; do
  if [[ "$FILE_PATH" == *"$p"* ]]; then
    echo "Blocked: $FILE_PATH matches protected pattern '$p'" >&2
    exit 2
  fi
done

# 2. Content-based block (very rough — tighten for production)
if echo "$CONTENT" | grep -Eiq '(AKIA[0-9A-Z]{16}|sk-[A-Za-z0-9]{20,}|-----BEGIN [A-Z ]*PRIVATE KEY-----)';
then
  echo "Blocked: content appears to contain a credential" >&2
  exit 2
fi
  exit 0
```

```
chmod +x .claude/hooks/block-secrets.sh
```

Step 2 — .claude/settings.json

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/block-secrets.sh"
          }
        ]
```

```
    }
   ],
   "PostToolUse": [
    {
     "matcher": "Edit|Write",
     "hooks": [
      {
       "type": "command",
       "command": "f=$(jq -r '.tool_input.file_path'); if [[ \"$f\" =~ \\.(js|ts|json|jsx|tsx|css|html|md)$ ]]; then
npx prettier --write \"$f\"; fi",
       "shell": "bash"
      }
     ]
    },
    {
     "matcher": "Powershell | Bash",
     "hooks": [
      {
       "type": "command",
       "command": "jq -c '{ts: now|todate, cmd: .tool_input.command, cwd: .cwd}' >>
\"$CLAUDE_PROJECT_DIR\"/.claude/bash-audit.log"
      }
     ]
    }
   ],
   "SessionStart": [
    {
     "matcher": "compact",
     "hooks": [
      {
       "type": "command",
       "command": "echo 'Project rules: TypeScript strict mode. Tests required for public APIs. No new
dependencies without approval.'"
      }
     ]
    }
   ]
```

```
  }
}
```

Step 3 — Verify

Inside a Claude Code session in the project:

```
/hooks


# You should see:
#  PreToolUse  (1)
#  PostToolUse (2)
#  SessionStart (1)
```

Step 4 — Try to break it

- Ask Claude to edit .env. The PreToolUse hook blocks it; Claude reads the stderr message and adapts.
- Ask Claude to create a HTML file (Put all HTML in one line)
- Ask Claude to run a Bash command. Check .claude/bash-audit.log — every command is logged with timestamp and cwd.
- Trigger compaction (long conversation or /compact). On the next turn the project rules are automatically reinjected.

Final thought

Hooks are how Claude Code stops being a clever assistant and starts being a controllable tool. Skills suggest what should happen; hooks guarantee it. Start with a notification hook to feel the rhythm, then add formatters and protected-file blockers to your projects. Within a week your team's standards will be enforced automatically.